# Chapter 3:
# WEB DESIGN AND DEVELOPMENT FUNDAMENTALS

# Generic element

- Generic elements are HTML tags that do not have any specific meaning or function, but are used to group or style other elements.
- For example,  and  are generic elements that can be used to create containers or inline elements with different classes or ids. Generic elements are more flexible and versatile than semantic elements, as they can be used for any purpose and customized with CSS.

# Generic element

- (x)html provides two generic elements (Div and Span) that can be used to describe your content perfectly.
- The difference between span and div is that a span element is *in-line* and usually used for a small chunk of HTML inside a line (such as inside a paragraph) where as a div (division) element is *block-line* (which is basically equivalent to having a line-break before and after it) and used to group larger chunks of code.
- You can give a generic element a name using either an id or class attribute.

# The div element

- Div used to brake a page into sections for context, structure, and layout purposes.
- The div (short for division) is used to indicate a generic block-level element
- The HTML  tag is used for defining a section of your document.
- With the div  tag, you can group large sections of HTML elements together and format them with CSS.

# The div element

Heading and several paragraph are enclose in a div and identified as the “news” section.

```
<html>  <head>   <title>Moby-Dick</title>
 <style type="text/css">
    #news {color: #ff0000}
  </style>
</head>
<body>
<div id=”news”>
<h1>New this week</h1>
<p>we’ve been working on…</p>
<p>and last but no least,…</p>
</div> </body></html>
```

**Output**

# The span Element

- The span element is used to indicate a generic inline element
- The span element is the generic inline element.

e.g.
```
<html>  <head>
    <title>Moby-Dick</title>
    <style type="text/css">
      span.html {color: #ff0000}
    </style>
  </head>
  <body>
  <p> HTML is really a very simple language
  <span class=”html”>It consists of ordinary text,</span> with commands that are enclosed by "<" and ">" characters </p>
  </body>
```

**Output**

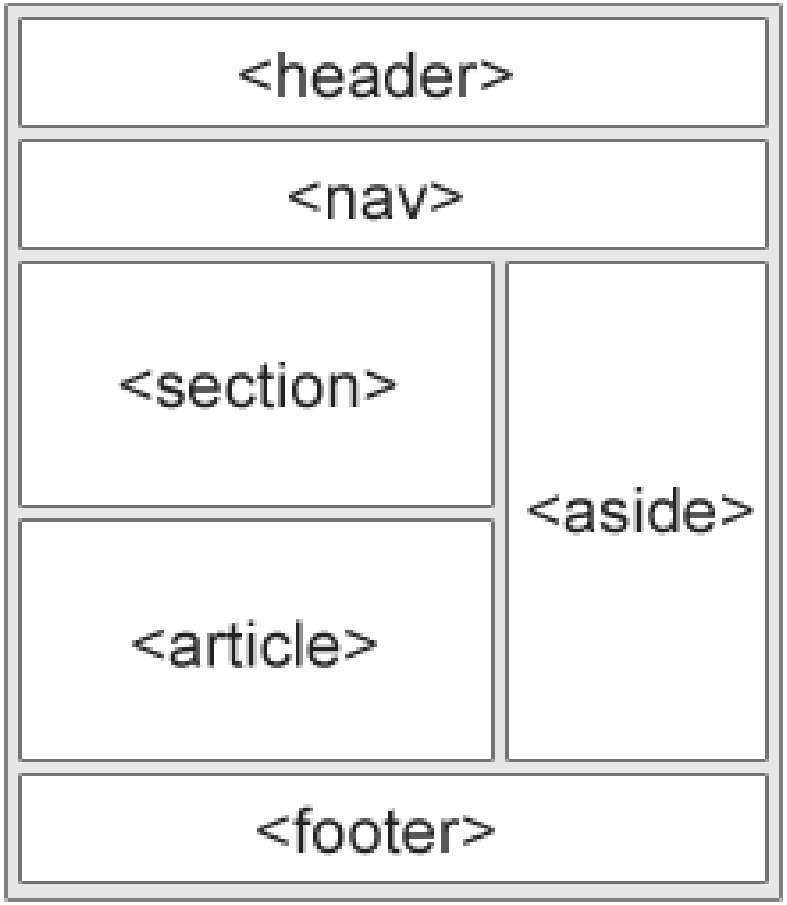
HTML is really a very simple language It consists of ordinary text, with commands that are enclosed by "<" and ">" characters

# Semantic elements

- Semantic elements are HTML tags that describe the content or function of the element, rather than just its appearance.
- For example, <header>, <nav>, <article>, <section>, <footer>, and <aside> are semantic elements that indicate the role of different parts of a web page.
- Semantic elements help improve the readability, accessibility, and **SEO** of your web pages, as they provide more information to browsers, screen readers, and search engines.
- **Search engine optimization (SEO)** corresponds to any steps to boost the number and desirability of consumers who arrive at a website through search engine result pages.

# Semantic elements

- The following example show semantic element layout by w3schools



# Semantic elements

- Elements such as <header>, <nav>, <section>, <article>, <aside>, and <footer> act more or less like  elements. They group other elements together into page sections. However where a  tag could contain any type of information, it is easy to identify what sort of information would go in a semantic <header> region.

# Search engine optimization

- There are two types of SEO

1) **On Page SEO**: involves optimizing different webpage components to make it look more engine-friendly. This incorporates optimizing meta labels (**title**, description), **headings**, **URL structure**, **inner connecting**, and **guaranteeing watchword significance** inside the content.

2) **Off page SEO**: Off-page SEO refers to all the activities you do outside your website to improve its ranking in search engine results. It focuses on increasing your site's authority, trustworthiness, and relevance by earning backlinks, social signals, and brand mentions.

# Element identifier

- There are two element identifiers **id** and **class** used to give names to the generic **div** and **span** element
- The id is used to identify a unique element in the document.
- **The id identifier**
  - The id identifier is used to identify a unique element in the document
  - The value of id must be used only once In the HTML document

# Element identifier

- **The class identifier**
  - The class attribute is used for grouping similar elements
  - they can be used multiple times in the same HTML document
- **Id and class value**
  - the values for id and class attribute should start with a letter (A-Z or a-z) or underscore
  - they should not contain any character spaces or special characters
  - Letter, number, hyphens, underscore, colons, and periods are allowed.

# Character entity

- Some characters have a special meaning in HTML, like the less than sign (<) that defines the start of an HTML tag .if want the browser to actually display these characters we must insert character entities in the HTML source.
- A character entity has three parts: an ampersand (&), an entity name or a # and an entity number and finally a semicolon (;)

# The most common character entity

| Result | description | Entity name | Entity number |
|---|---|---|---|
| | Non-braking space |   |   |
| < | Less than | &lt; | &#60; |
| > | Greater than | &gt; | &#62; |
| “ | Quotation mark | &quot; | &#34; |
| ‘ | apostrophe | &apos; | &#39; |
| ¢ | cent | &cent; | &#162 |
| £ | pound | &pound; | &#163; |
| ¥ | yen | &yen; | &#165; |
| © | copyright | &copy; | &#169; |
| ® | Registerd trademark | &reg; | &#174; |
| × | Multiplacation | &times; | &#215; |
| ÷ | division | &divide; | &#245; |

# HTML Tables

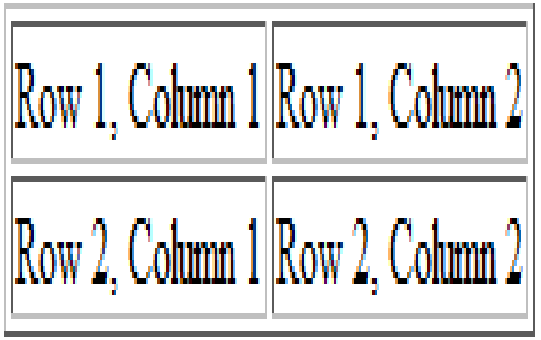
- Tables are very useful to arrange in HTML and they are used very frequently by almost all web developers.
- Tables are just like spreadsheets and they are made up of rows and columns.
- You will create a table in HTML/XHTML by using <table> tag. Inside <table> element the table is written out row by row.
- A row is contained inside a <tr> tag . which stands for table row. And each cell is then written inside the row element using a <td> tag . which stands for table data.

# HTML Tables cont...

- Example

```
<HTML>
<HEAD>
<TITLE>Table</TITLE>
</HEAD>
<BODY>
<table border="1">
<tr> <td>Row 1, Column 1</td>
<td>Row 1, Column 2</td> </tr>
<tr> <td>Row 2, Column 1</td>
<td>Row 2, Column 2</td> </tr> </table>
</BODY>
</HTML>
```

***output***




**NOTE: In the above example *border is an attribute of <table> and it will put border across all the cells. If you do not need a border then you can use border="0".***

# HTML Tables cont...

Table Heading - The <th> Element:

- Table heading can be defined using <th> element. This tag will be put to replace <td> tag which is used to represent actual data.
- Normally you can put your top row as table heading as shown below, otherwise you can use <th> element at any place:

# HTML Tables cont...

- Example:

```
<table border="1">
<tr>
<th>Name</th>
<th>Salary</th>
</tr>
```

***Output***

| Name | Salary |
| --- | --- |
| Marta Belachew | 5000 |
| Belachew Tomass | 7000 |

# Table Cellpadding and Cellspacing:

- There are two attributes called cellpadding and cellspacing which use to adjust the white space in your table cell.
- Cellspacing defines the width of the border, while cellpadding represents the distance between cell borders and the content within. Following is the example:

# Table Cellpadding and Cellspacing(Cont…)

- Example

```
<HTML>
<HEAD>
<TITLE>Table </TITLE>
</HEAD>
<BODY>
<table border="1" cellpadding="5" cellspacing="5">
<tr> <th>Name</th> <th>Salary</th> </tr>
<tr> <td>Marta Belachew</td> <td>5000</td> </tr>
<tr> <td>Belachew Tomass</td> <td>7000</td> </tr>
</table>
</BODY>
</HTML>
```

**Output**

| Name | Salary |
|---|---|
| Marta Belachew | 5000 |
| Belachew Tomass | 7000 |

# Colspan and Rowspan Attributes:

- You can use colspan attribute if you want to merge two or more columns into a single column.
- Similar way you can use rowspan if you want to merge two or more rows. Following is the example:

```
<table border="1">
<tr>
<th>Column 1</th>
<th>Column 2</th>
<th>Column 3</th>
</tr>
<tr><td rowspan="2">Row 1 Cell 1</td>
<td>Row 1 Cell 2</td><td>Row 1 Cell 3</td></tr>
<tr><td>Row 2 Cell 2</td><td>Row 2 Cell 3</td></tr>
<tr><td colspan="3">Row 3 Cell 1</td></tr>
</table>
```

Output

| Column 1 | Column 2 | Column 3 |
|---|---|---|
| Row 1 Cell 1 | Row 1 Cell 2 | Row 1 Cell 3 |
| | Row 2 Cell 2 | Row 2 Cell 3 |
| Row 3 Cell 1 | | |

# Tables Backgrounds

- You can set table background using the following two ways:

✓ Using *bgcolor attribute - You can set background color for whole table or just for one cell.*

✓ Using *background attribute - You can set background image for whole table or just for one cell.*

- NOTE:You can set border color also using ***bordercolor*** *attribute.*
- Here is an example of using *bgcolor attribute:*

- Here is an example of using *bgcolor attribute:*

```
<table border="5" bordercolor="green" bgcolor="gray">
 <tr>
      <th>Column 1</th> <th>Column 2</th>
      <th>Column 3</th> </tr>
<tr>
      <td rowspan="2">Row 1 Cell 1</td>
       <td bgcolor="red">Row 1 Cell 2</td><td>Row 1 Cell 3</td>
</tr>
 <tr>
      <td>Row 2 Cell 2</td>
      <td>Row 2 Cell 3</td>
</tr>
 <tr>
      <td colspan="3">Row 3 Cell 1</td>
</tr>
 </table>
```

**Output**

Here is an example of using bgcolor attribute:

| Column 1 | Column 2 | Column 3 |
|---|---|---|
| Row 1 Cell 1 | Row 1 Cell 2 | Row 1 Cell 3 |
| | Row 2 Cell 2 | Row 2 Cell 3 |
| Row 3 Cell 1 | | |

# Frame

- HTML frames are used to divide your browser window into multiple sections where each section can load in a separate HTML document.
- A collection of frames in the browser window is known as a frameset.
- The window is divided into frames in a similar way the tables are organized: *into rows and columns.*

# Frame(Cont…)

**Creating Frames**

- To use frames on a page we use <frameset> tag instead of <body> tag. The <frameset> tag defines how to divide the window into frames.
- The **rows attribute of <frameset> tag defines** horizontal frames and **cols attribute defines vertical frames.**
- **Each frame is indicated by <frame>** tag and it defines which HTML document shall open into the frame.

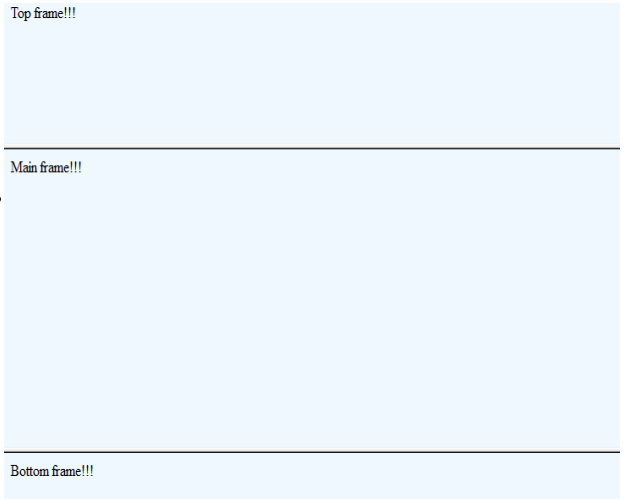
# Frame(Cont…)

**Example**

- Following is the example to create three horizontal frames:

```
<html><head>
<title>HTML Frames</title></head>
<frameset rows="25%,50%,25%">
<frame name="Top" src="Top.html">
<frame name="main" src="main.html">
<frame name="bottom" src="bottom.html">
<noframes>
<body>
Your browser does not support frames.
</body>
</noframes>
</frameset></html>
```

**Output**



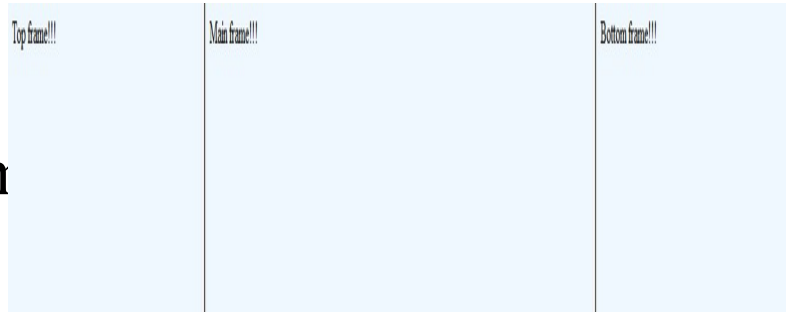
# Frame(Cont…)

## Example

- Let's put above example as follows, here we replaced rows attribute by cols and changed their width. This will create all the three frames vertically:

```
<html><head><title>HTML Frames</title>
</head>
<frameset cols="25%,50%,25%">
<frame name="Top" src="Top.html">
<frame name="main" src="main.html">
<frame name="bottom " src="bottom.html">
<noframes>
<body>
Your browser does not support fran
</body>
</noframes></frameset></html>
```

**Output**



# The <frameset> Tag Attributes

- Following are important attributes of the <frameset> tag:
- **Cols**: specifies how many columns are contained in the frameset and the size of each column. You can specify the width in one of four ways:
  - Absolute value in pixels. For example to crate three vertical frames , use cols="100,500,100".
  - A percentage of the browser window. For example to crate three vertical frames, use cols="10%,80%,10%".
  - Using a wildcard symbol. For example to create three vertical frames ,use cols="10%,*,10%". In this case wildcard takes remainder of the window.
  - Relative width of browser window. For example to create three vertical frames, use cols="3*,2*,1*".this is an alternative to percentages. You can use relative widths of the browser window. Here the window is divided into sixths: the first column takes up half of the window, the second takes one third and the third takes one sixth.

# The <frameset> Tag Attributes

- **Rows** : This attribute works just like the cols attribute and takes the same value. But it used to specify the rows in the frameset. For example to create two horizontal frames, use rows="10%,90%". You can specify the height of each row in the same way as explained above for columns.
- **Border** : this attribute specifies the width of the border of each frame in pixels. for example border="5". A value of zero means no border.
- **Framespacing** : this attribute specifies the amount of space between frames in a frameset. It can take any integer value. For example framespaceing="10" means there should be 10 pixel spacing between each frames.

# The <frame> Tag Attributes

- Following are important attributes of <frame> tag:
  - **Src**: this attribute is used to give the file name that should be loaded in the frame. Its value can be any URL.
  - **Name**: this attribute allows you to give a name to a frame. It is used to indicate which frame a document should be loaded into. This is specially important when you want to create links in one frame that load pages into another frame. in this case the second frame needs a name to identify itself as the target of the link.
  - **frameborder**: This attribute specifes whether or not the borders of that frame are shown; it overrides the value given in the frame border attribute on the <frameset> tag if one is given, and this can take values either 1 yes 0 no.
  - **Marginwidth**: this attribute allows you to specify the width of the space between the left and right of the frame's borders and the frame's content. The value is given in pixels. For example marginwidth="10".

# The <frame> Tag Attributes

- **Marginheight**: this attribute allows you to specify the height of the space between the top and bottom of the frame's borders and its contents. The value is given in pixels. For example marginheight="10".
- **Noresize**: by default you can resize any frame by clicking and dragging on the borders of a frame. The noresize attribute prevents a user from being able to resize the frame.
- **Scrolling**: this attribute controls the appearance of the scrollbars that appear on the frame. This takes values either "yes","no" or "auto". for example scrolling="no" means it should not have scroll bars.

# Browser Support for Frames

- If a user is using any old browser or any browser which does not support frames then <noframes> element should be displayed to the user.
- So you must place a <body> element inside the <noframes> element because the <frameset> element is supposed to replace the <body> element,
- But if a browser does not understand <frameset> element then it should understand what is inside the <body> element which is contained in a <noframes> element.

# Frame's name and target attributes

- One of the most popular uses of frames is to place navigation bars in one frame and then load main pages into a separate frame.
- Let's see next slide example.

✓ ***The content of index.html file***

```
<html><head>
<title>HTML Frames</title>
</head>
<frameset cols="25%,75%" border="6">
<frame
name="menu_page"src="menu.html">
<frame name="main_page"src="main.html">
<noframes>
<body>
Your browser does not support frames.
</body>
</noframes>
</frameset>
</html>
```

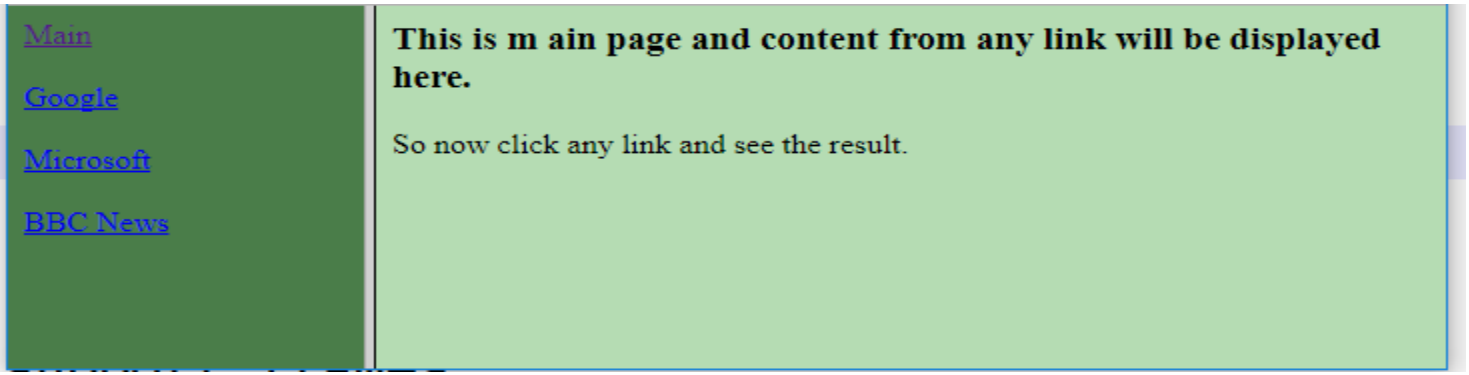
✓ The content of main.html file:

```
<htm l>
<body bgcolor="#b5dcb3">
<h3>This is m ain page and content from any
link will be displayed here.</h3>
<p>So now click any link and see the
result.</p>
</body></htm l>
```

✓***The content of menu.html file***

```
<html>
<body bgcolor="#4a7d49">
<a href="main.html"
target="main_page">main</a><br/>
<a href="http://www.google.com
"target="main_page">Google</a>
<br /><br />
<a href="http://www.m icrosoft.com
"target="main_page">Microsoft</a>
<br /><br />
<a
href="http://news.bbc.co.uk"target="main_pa
ge">BBC News</a>
</body>
</html>
```

# Output




Now you can try to click links available in the left panel and see the result. The *target* attribute can also take one of the following values:

| Option | Description |
|---|---|
| _self | Loads the page into the current frame. |
| _blank | Loads a page into a new browser window.opening a new window. |
| _parent | Loads the page into the parent window, which in the case of a single frameset is the main browser window. |
| _top | Loads the page into the browser window, replacing any current frames. |
| targetframe | Loads the page into a named targetframe. |

# Disadvantages of Frames

- There are few drawbacks with using frames, so it's never recommended to use frames in your Web pages:
  - Some smaller devices cannot cope with frames often because their screen is not big enough to be divided up.
  - Sometimes your page will be displayed differently on different computers due to different screen resolution.
  - The browser's *back button might not work as the user hopes*.
  - There are still few browsers that do not support frame technology.

# HTML Forms

- HTML Forms are required when you want to collect some data from the site visitor. For example registration information: name, email address, credit card, etc.
- A form will take input from the site visitor and then will post it to the back-end application such as CGI, ASP Script or PHP script etc. Then the back-end application will do required processing on that data in whatever way you like.
- The <form> element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

# Creating a form

- In an HTML document, forms are set between the <FORM> container tags. The form container works as follows:

<FORM name ="name_of_the_form" METHOD="how_to_send" ACTION="URL of script"> ...form data... </FORM>

- Most frequently used form attributes are:
  - ✓ **name**: This is the name of the form.
  - ✓ **Action**: Here you will specify any script URL which will receive uploaded data. which simply accepts the URL for the script that will process the data from your form.

# Forms (Cont.…)

- **Target**: It specifies the target page where the result of the script will be displayed. It takes values like _blank, _self, _parent etc.
- **Method**: Here you will specify method to be used to upload data. It can take various values but most frequently used are GET and POST.
  - Get: is best used with single responses, like a single textbox.
  - Post: it is better for a greater amount of data to be sent

# Forms (Cont.…)

- The GET Method
  - query strings (name/value pairs) is sent in the URL of a GET request:

  /test/demo_form.asp?name1=value1&name2=value2
  - Some other notes on GET requests:
    - GET requests remain in the browser history
    - GET requests should never be used when dealing with sensitive data
    - GET requests have length restrictions
    - GET requests should be used only to retrieve data
    - POST requests can be bookmarked
- The POST Method
  - query strings (name/value pairs) is sent in the HTTP message body of a POST request:
  - POST /test/demo_form.asp HTTP/1.1 Host: w3schools.com name1=value1&name2=value2
  - Some other notes on POST requests:
    - POST requests do not remain in the browser history
    - POST requests cannot be bookmarked
    - POST requests have no restrictions on data length

# Forms (Cont.…)

- There are different types of form controls that you can use to collect data from a visitor to your site.
- The HTML <form> element can contain one or more of the following form elements:
  - <input>
  - <label>
  - <select>
  - <textarea>
  - <button>
  - <fieldset>
  - <legend>
  - <datalist>
  - <output>
  - <option>
  - <optgroup>

## The <input> Element

- One of the most used form elements is the <input> element.
- The <input> element can be displayed in several ways, depending on the type attribute.

```
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname">
```

## The <label> Element

- The <label> element defines a label for several form elements.
- The <label> element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focus on the input element.

# HTML Forms - Text Input Controls:

- There are actually three types of text input used on forms:

✓ Single-line text input controls: Used for items that require only one line of user input, such as search boxes or names. They are created using the <input> element.

✓ Password input controls: Single-line text input that mask the characters a user enters.

✓ Multi-line text input controls: Used when the user is required to give details that may be longer than a single sentence. Multi-line input controls are created with the <textarea> element.

# Single-line text input controls:

- Single-line text input controls are created using an <input> element whose type attribute has a value of text.
- Here is a basic example of a single-line text input used to take first name and last name:

```
<form action="" method="get">
First name:
<input type="text" name="first_name" />
<br>
Last name:
<input type="text" name="last_name" />
<input type="submit" value="submit" />
</form>
```

**Output**

# <input> tag attributes

- Following is the list of attributes for <input> tag.

✓ **type**: Indicates the type of input control you want to create. This element is also used to create other form controls such as radio buttons and checkboxes.

✓ **name**: Used to give the name part of the name/value pair that is sent to the server, representing each form control and the value the user entered.

✓ **value**: Provides an initial value for the text input control that the user will see when the form loads.

✓ **size**: Allows you to specify the width of the text-input control in terms of characters.

✓ **maxlength**: Allows you to specify the maximum number of characters a user can enter into the text box.

# Password input controls:

- This is also a form of single-line text input controls are created using an <input> element whose type attribute has a value of password.
- Here is a basic example of a single-line password input used to take user password:

```
<form action=" " method="get">
Login :<input type="text" name="login" /><br>
Password:<input type="password"
   name="password" />
<input type="submit" value="submit" />
</form>
```

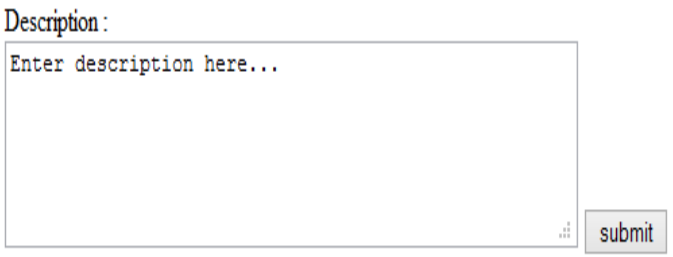
Output

# Multiple-Line Text Input Controls:

- If you want to allow a visitor to your site to enter more than one line of text, you should create a multiple-line text input control using the <textarea> element.
- Here is a basic example of a multi-line text input used to take item description:

```
<HTML><HEAD><TITLE>Table</TITLE></HEAD>
<BODY>
<form action="/cgi-bin/hello_get.cgi" method="get">
Description : <br />
<textarea rows="5" cols="50" name="coment">
Enter description here...
</textarea>
<input type="submit" value="submit"/>
</form></BODY></HTML>
```

**Output**



# <textarea> tag attributes

- Following is the detail of above used attributes for <textarea> tag.
  - ✓ name: The name of the control. This is used in the name/value pair that is sent to the server.
  - ✓ rows: Indicates the number of rows of text area box.
  - ✓ cols: Indicates the number of columns of text area box.

# HTML Forms - Creating Button:

- There are various ways in HTML to create clickable buttons. You can create clickable button using <input> tag.
- When you use the <input> element to create a button, the type of button you create is specified using the type attribute. The type attribute can take the following values:

✓ submit: This creates a button that automatically submits a form.

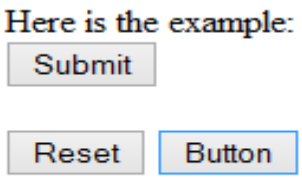
✓ reset: This creates a button that automatically resets form controls to their initial values.

✓ button: This creates a button that is used to trigger a client-side script when the user clicks that button.

# Here is the example:

```
<form action="http://www.example.com/test.asp" method="get">
 <input type="submit" name="Submit" value="Submit" />
 <br /><br />
 <input type="reset" value="Reset" />
 <input type="button" value="Button" />
 </form>
```

**Output**



# HTML Forms - Checkboxes Control:

- Checkboxes are used when more than one option is required to be selected. They are created using <input> tag as shown below.
- Here is example HTML code for a form with two checkboxes

```
<form action="" method="get">
<input type="checkbox" name="subject" value="maths"
  checked > Maths
<input type="checkbox" name="subject " value="physics">
  Physics
 </form>
```

**Output**

# Checkbox attributes

- Following is the list of important checkbox attributes:

✓ type: Indicates that you want to create a checkbox.

✓ name: Name of the control. More than one checkbox should share the same name only if you want to allow users to select several items from the same list.

✓ value: The value that will be used if the checkbox is selected.

✓ checked: Indicates that when the page loads, the checkbox should be selected.

# HTML Forms - Raidobox Control:

- Radio Buttons are used when only one option is required to be selected. They are created using <input> tag as shown below:
- Here is example HTML code for a form with two radio button:

```
<form action="" method="post">
<input type="radio" name="subject" value="maths" />
   Maths
<input type="radio" name="subject" value="physics" />
   Physics
</form>
```

**Output**

# Select

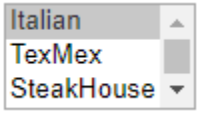
- The **HTML <select> element** represents a control that provides a menu of options:

<SELECT NAME="variable"> <OPTION SELECTED VALUE="value"> Menu text <OPTION VALUE="value"> Menu text ... </SELECT

Example

Choose your favorite food:

```
<SELECT NAME="food">
 <OPTION SELECTED VALUE="ital"> Italian</option>
 <OPTION VALUE="texm"> TexMex </option>
 <OPTION VALUE="stek"> SteakHouse</option>
 <OPTION VALUE="chin"> Chinese </option>
 </SELECT>
```




- **Attributes**
  - **SELECTED**: attribute is simply designed to show which value will be the default in the menu listing.
  - **VALUE:** can be anything you want to pass on to the Web server and associated script for processing.

# Select

- **Size:** attribute to specify the number of visible values:
- **multiple** attribute to allow the user to select more than one value

```
<label for="cars">Choose a car:</label>
<select id="cars" name="cars" size="4" multiple>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

# HTML <optgroup> Tag

- The <optgroup> tag is used to group related options in a <select> element (drop-down list)

```
<label for="cars">Choose a car:</label>
<select  name="cars" id="cars">
 <optgroup label="Swedish Cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
 </optgroup>
 <optgroup label="German Cars">
  <option value="mercedes">Mercedes</option>
  <option value="audi">Audi</option>
 </optgroup>
</select>
```

# HTML <datalist> Tag

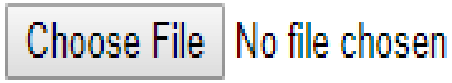
- The <datalist> tag specifies a list of pre-defined options for an <input> element.
- The <datalist> tag is used to provide an "autocomplete" feature for <input> elements. Users will see a drop-down list of pre-defined options as they input data.
- The <datalist> element's id attribute must be equal to the <input> element's list attribute (this binds them together).

```
<label for="browser">Choose your browser from the list:</label>
<input list="browsers" name="browser" id="browser">
<datalist id="browsers">
  <option value="Edge">
  <option value="Firefox">
  <option value="Chrome">
  <option value="Opera">
  <option value="Safari">
</datalist>
```

**File select boxes**

- <input> elements with **type="file"** let the user choose one or more files from their device storage.
- Example

  <input name="myFile" type="file">



# The <fieldset> and <legend> Elements

- The **<fieldset>** element is used to group related data in a form.
- The **<legend>** element defines a caption for the <fieldset> element.

```
<form action="/action_page.php">
  <fieldset>
    <legend>Personalia:</legend>
    <label for="fname">First name:</label><br>
    <input type="text"  name="fname" value="Hana"><br>
    <label for="lname">Last name:</label><br>
    <input type="text"  name="lname" value="Sisay"><br><br>
    <input type="submit" value="Submit">
  </fieldset>
</form>
```

# HTML Comments

- Comments are piece of code which is ignored by any web browser.
- It is good practice to comment your code, especially in complex documents, to indicate sections of a document, and any other notes to anyone looking at the code.
- Comments help you and others understand your code.

# HTML Comments (Cont...)

- HTML Comment lines are indicated by the special beginning tag <!-- and ending tag --> placed at the beginning and end of EVERY line to be treated as a comment.
- Comments do not nest, and the double-dash sequence "--" may not appear inside a comment except as part of the closing --> tag.
- You must also make sure that there are no spaces in the start-of-comment string.

For example: Given line is a valid comment in HTML

```
<!--   This is commented out -->
```

# Multiline Comments:

- You have seen how to comment a single line in HTML.
- You can comment multiple lines by the special beginning tag <!-- and ending tag --> placed before the first line and end of the last line to be treated as a comment.

For example:

```
<!--
This is a multiline comment <br />
and can span through as many as lines you like.
-->
```

# Question ?